

Projection Method for Incompressible Flow: Report-Code Mapping

1. Introduction

This report presents a numerical solution of the two-dimensional incompressible Navier-Stokes equations using the **projection method**. The objective is not only to describe the mathematical formulation, but also to establish a **direct, line-by-line correspondence** between the governing equations and the implemented Python code (`projection_method_2D.py`).

2. Governing Equations

The incompressible Navier-Stokes equations are given by:

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla) \mathbf{u} = -\frac{1}{\rho} \nabla p + \nu \Delta \mathbf{u},$$
$$\nabla \cdot \mathbf{u} = 0.$$

Here, $\mathbf{u} = (v, w)$ denotes the velocity field, p the pressure, ρ the density, and ν the kinematic viscosity.

3. Spatial and Temporal Discretization

The computational domain is defined as:

$$(y, z) \in [0, 1] \times [0, 1]$$

A uniform Cartesian grid is used with spacings Δy and Δz . Time is discretized using a **forward Euler scheme** with time step Δt .

Code Reference

```
Ny, Nz = 50, 50
Ly, Lz = 1.0, 1.0

dy = Ly / (Ny - 1)
dz = Lz / (Nz - 1)

dt = 0.001
nt = 100
```

4. Initial Conditions

The flow is initially at rest throughout the domain:

$$v(y, z, 0) = 0, \quad w(y, z, 0) = 0.$$

The pressure field is initialized to zero.

Code Reference

```
v = np.zeros((Ny, Nz))
w = np.zeros((Ny, Nz))
p = np.zeros((Ny, Nz))
```

5. Finite Difference Operators

Second-order central finite differences are used for all spatial derivatives.

Divergence Operator

$$\nabla \cdot \mathbf{u} = \frac{\partial v}{\partial y} + \frac{\partial w}{\partial z}$$

Code Reference

```
def divergence(v, w):
    ...
```

Gradient Operator

$$\nabla p = \left(\frac{\partial p}{\partial y}, \frac{\partial p}{\partial z} \right)$$

Code Reference

```
def gradient(p):
    ...
```

Laplacian Operator

$$\Delta f = \frac{\partial^2 f}{\partial y^2} + \frac{\partial^2 f}{\partial z^2}$$

Code Reference

```
def laplacian(f):  
    ...
```

6. Predictor Step (Intermediate Velocity)

In the predictor step, the momentum equation is solved **without the pressure term**:

$$\mathbf{u}^* = \mathbf{u}^n + \Delta t \left[-(\mathbf{u}^n \cdot \nabla) \mathbf{u}^n + \nu \Delta \mathbf{u}^n \right].$$

This step produces an intermediate velocity field that does not satisfy the incompressibility constraint.

Code Reference

```
def predictor_step(v, w):  
    ...  
    v_star = v + dt * (-conv_v + diff_v)  
    w_star = w + dt * (-conv_w + diff_w)
```

7. Pressure Poisson Equation

To enforce incompressibility, the pressure is obtained from the Poisson equation:

$$\Delta p^{n+1} = \frac{\rho}{\Delta t} \nabla \cdot \mathbf{u}^*.$$

Homogeneous Neumann boundary conditions are applied:

$$\frac{\partial p}{\partial n} = 0.$$

Code Reference

```
rhs = (rho / dt) * divergence(v_star, w_star)  
p = pressure_poisson(p, rhs)
```

8. Projection Step (Velocity Correction)

The velocity field is corrected using the pressure gradient:

$$\mathbf{u}^{n+1} = \mathbf{u}^* - \frac{\Delta t}{\rho} \nabla p^{n+1}.$$

This step ensures that the updated velocity field is divergence-free.

Code Reference

```
v, w = projection_step(v_star, w_star, p)
```

9. Boundary Conditions

The following boundary conditions are applied: - Periodic boundary conditions in the y-direction - No-penetration walls at $z = 0$ and $z = 1$ ($w = 0$)

Code Reference

```
def apply_velocity_bc(v, w):  
    ...
```

10. Time Integration Loop

The full numerical algorithm is executed within a time-stepping loop:

$$\mathbf{u}^n \rightarrow \mathbf{u}^* \rightarrow p^{n+1} \rightarrow \mathbf{u}^{n+1}$$

Code Reference

```
for n in range(nt):  
    v_star, w_star = predictor_step(v, w)  
    rhs = (rho / dt) * divergence(v_star, w_star)  
    p = pressure_poisson(p, rhs)  
    v, w = projection_step(v_star, w_star, p)  
    v, w = apply_velocity_bc(v, w)
```

11. Conclusion

This study demonstrates a complete and consistent numerical implementation of the projection method for incompressible flows. Each mathematical equation in the governing formulation has a clear and explicit counterpart in the Python code. This one-to-one correspondence ensures transparency, reproducibility, and suitability for academic reporting, coursework, and introductory CFD research.